

טכנולוגיות אינטרנט מתקדמות - 61776 (WEB)

הגשת פרויקט

B20

BIO-BOT 2.0

שם חברת הצוות	תז
ניקולאי סבצ'נקו	REDACTED
נתן נימיץ	REDACTED
אופיר דגן	REDACTED
לירון זינו	REDACTED
רון קריספין	REDACTED

BIO-BOT 2.0 הוא צ'אטבוט בעברית שעוזר לסטודנטים להנדסת ביוטכנולוגיה במכללת בראודה. הסטודנט בוחר שנתון, שואל שאלה בשפה חופשית כמו שהוא היה שואל חבר, והבוט מחזיר תשובה ברורה ומעוצבת. מאחורי הקלעים המערכת משלבת לוגיקה קבועה ומדויקת עם מודלים של בינה מלאכותית שמבינים את כוונת השאלה ומאתרים את התשובה המתאימה בעזרת שימוש בכלים ומנוע RAG.

קישורים:

[סרטון שימוש מערכת](#)

[קישור לגיט](#)

מה הסטודנט יכול לעשות:

יכול לשאול על קורסים ותנאי קדם, לתרגם שם קורס למספר ולהפך, לגלות אילו קורסים הוא חייב לפני קורס מסוים (כולל שרשרת שלמה של תנאים), ולבדוק אם אפשר לקחת שני קורסים במקביל או מה קודם למה. הוא יכול לקבל מידע על הרשמה לקורסים, חלונות הרשמה, דרישות נ"ז, יועצים וחונכים, וגם על פטורים והרשמה חריגה. יש מערכת מעבדות שמראה תאריכים, ימים, שעות, קבוצות ומרצים לפי סמסטר. סטודנט במילואים מקבל תהליך מודרך שמתאים לו את ההתאמות לפי הזכאות שלו. אפשר גם לאתר יועץ לפי סמסטר, ומסמסטר 5 ומעלה גם לפי התמחות. אם הסטודנט כותב משהו שמשדר מצוקה, הבוט לא מנסה לתת תשובה עובדתית אלא מגיב ברגישות ומפנה אותו לדייקנט הסטודנטים. בנוסף הוא יכול להוריד טפסים רשמיים ולהשאיר משוב אנונימי על התשובה שקיבל.

מה שיש למנהל:

למנהל יש פאנל מוגן בסיסמה שממנו הוא שולט בכל התוכן. הוא מעלה קובצי שנתון או מערכת מעבדות והמערכת מפרקת אותם אוטומטית לנתונים. הוא עורך קורסים, תנאי קדם, מעבדות, יועצים, הנחיות הרשמה וטפסים. הוא מנהל מאגר של תשובות מאושרות שהבוט מסתמך עליהן, רואה אילו שאלות נשאר בלי מענה

כדי להשלים אותן, וקורא את המשובים של הסטודנטים. יש גם לוח נתונים עם גרפים שמראה כמה שאלות נשאלו, מאיפה הגיעו התשובות ואיזה אחוז מהן נענה.

טכנולוגיות	חלק
React 19, Vite 7	צד לקוח (Frontend)
Lucide, Tailwind CSS 4, shadcn/ui, MUI X Charts, מצב בהיר וכהה	עיצוב (Styling)
Node.js, Express 5, שכבת שירותים מרכזית ללוגיקה	צד שרת (Backend)
Firebase Admin SDK דרך Google Cloud Firestore	בסיס נתונים (Database)
OpenAI: gpt-4o-mini להבנה וניסוח, text-embedding-3-small לאחזור סמנטי, Tool-calling router	בינה מלאכותית (AI)
Python: python-docx ליוורד, openpyxl לאקסל	פענוח מסמכים
JWT, bcrypt	אבטחה
Vercel ללקוח, Render לשרת, GitHub Actions ו-Brevo לדוח שבועי	פריסה ואוטומציה

קישור לתיקיית גיט: <https://github.com/SteveTrabajo/Web-Project>

קישור לאתר: <https://web-project-gules-sigma.vercel.app/>

2. רשימת דרישות

2.1 דרישות פונקציונליות (FR):

סטודנט:

- המערכת תאפשר לסטודנט לבחור שנתון.
- המערכת תאפשר לסטודנט לשאול שאלה בשפה חופשית ולקבל תשובה.
- המערכת תאפשר ניתוב השאלה ליכולת המתאימה לפי כוונתה.
- המערכת תאפשר תרגום שם קורס למספר קורס ולהפך.
- המערכת תאפשר הצגת תנאי קדם ישירים ורקורסיביים של קורס.
- המערכת תאפשר הצגת קורסים מקבילים של קורס נתון.
- המערכת תאפשר בדיקת היחס בין שני קורסים.
- המערכת תאפשר חיפוש הפוך של הקורסים הדורשים קורס נתון כתנאי קדם.
- המערכת תאפשר קבלת מידע על הרשמה לקורסים.
- המערכת תאפשר הצגת מערכת המעבדות לפי סמסטר.
- המערכת תאפשר לסטודנט במילואים לעבור תהליך מודרך לבחירת התאמות.
- המערכת תאפשר איתור יועץ לפי סמסטר והתמחות.
- המערכת תאפשר הפניית סטודנט במצוקה לדיקנט הסטודנטים.
- המערכת תאפשר צירוף טופס רלוונטי לתשובה והורדתו.
- המערכת תאפשר לסטודנט להשאיר משוב על תשובה.
- המערכת תאפשר שמירת שאלה שנשארה ללא מענה לטיפול מנהל.

מנהל

- המערכת תאפשר התחברות לפאנל הניהול ואיפוס סיסמה.
- המערכת תאפשר העלאת קובצי שנתון ומעבדות ופענוחם לנתונים.
- המערכת תאפשר ניהול קורסים ותנאי קדם.
- המערכת תאפשר ניהול מעבדות, יועצים, הנחיות הרשמה וטפסים.
- המערכת תאפשר ניהול מאגר התשובות המאושרות.
- המערכת תאפשר למנהל לצפות בשאלות ללא מענה.
- המערכת תאפשר למנהל לצפות במשובי הסטודנטים.
- המערכת תאפשר הצגת לוח נתונים עם גרפים על השימוש.
- המערכת תאפשר הרצת בדיקות איכות נתונים.
- המערכת תאפשר שליחת דוח שבועי בדוא"ל.

2.2 דרישות לא פונקציונליות (NFR):

- **ביצועים:** המערכת תשיב בזמן סביר. לפני כל קריאה גנרטיבית תבוצע קודם ניסיון אחזור מהיר מתוך התשובות המאפשרות.
- **מדרגיות:** הוספת יכולת חדשה תתאפשר בהוספת פונקציה ותיאור בלבד, בלי שכתוב שכבת הניתוב.
- **זמינות:** המערכת תרוץ בענן וזמינה מכל דפדפן.
- **אמינות:** לוגיקת התשובות תהיה דטרמיניסטית ונכונה, ונפרדת משכבת ההחלטה, כך ששינוי ניתוב לא ישבור אותה.
- **אבטחה:** הכניסה לניהול תוגן ב-JWT, הסיסמאות יוצפנו ב-bcrypt, תיבת השאלות תוגבל בקצב, גודל הבקשה יוגבל.
- **שמישות:** הממשק יהיה בעברית, פשוט לתפעול, יתמוך במצב בהיר וכהה ויציג תשובות קריאות ומעוצבות.
- **תחזוקתיות:** הקוד יאורגן בשכבת שירותים משותפת, ומתג הפעלה יאפשר חזרה מיידית למנגנון הניתוב הישן.
- **ניידות:** צד הלקוח וצד השרת יפרסו בנפרד, עם הגדרות סביבה חיצוניות למפתחות ולכתובות.
- **תאימות:** המערכת תעבוד בדפדפנים מודרניים ותותאם גם למובייל.

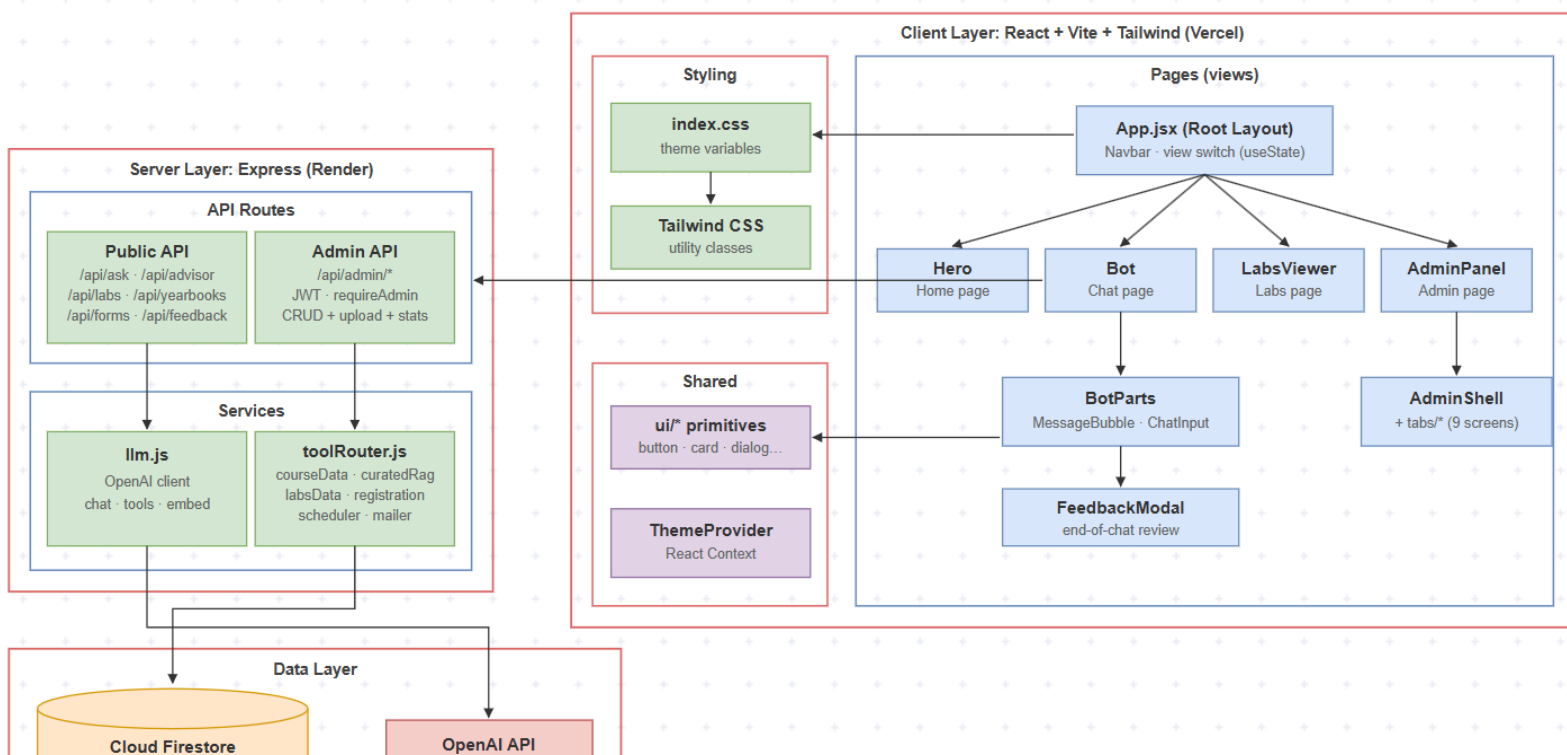
2.3 דרישות ממשק חיצוניות:

- **ממשקי משתמש:** ממשק צ'אט בעברית לסטודנט ופאנל ניהול בכרטיסיות למנהל, שניהם רספונסיביים ותומכים במצב בהיר וכהה.
- **ממשקי חומרה:** אין דרישות חומרה ייעודיות. המערכת נגישה מכל מכשיר עם דפדפן וחיבור לאינטרנט.
- **ממשקי תוכנה:** המערכת מתממשקת מול OpenAI API להבנה, ניסוח ואחזור סמנטי, מול Google Cloud Firestore לאחסון, מול Brevo HTTP API לשליחת דוא"ל, ומול סקריפטים ב-Python לפענוח מסמכי DOCX ו-Excel.
- **ממשקי תקשורת:** התקשורת בין הלקוח לשרת מתבצעת ב-HTTPS דרך ממשק REST. משימות מתוזמנות מופעלות דרך GitHub Actions הפונה לנקודת קצה פנימית מוגנת בסוד.

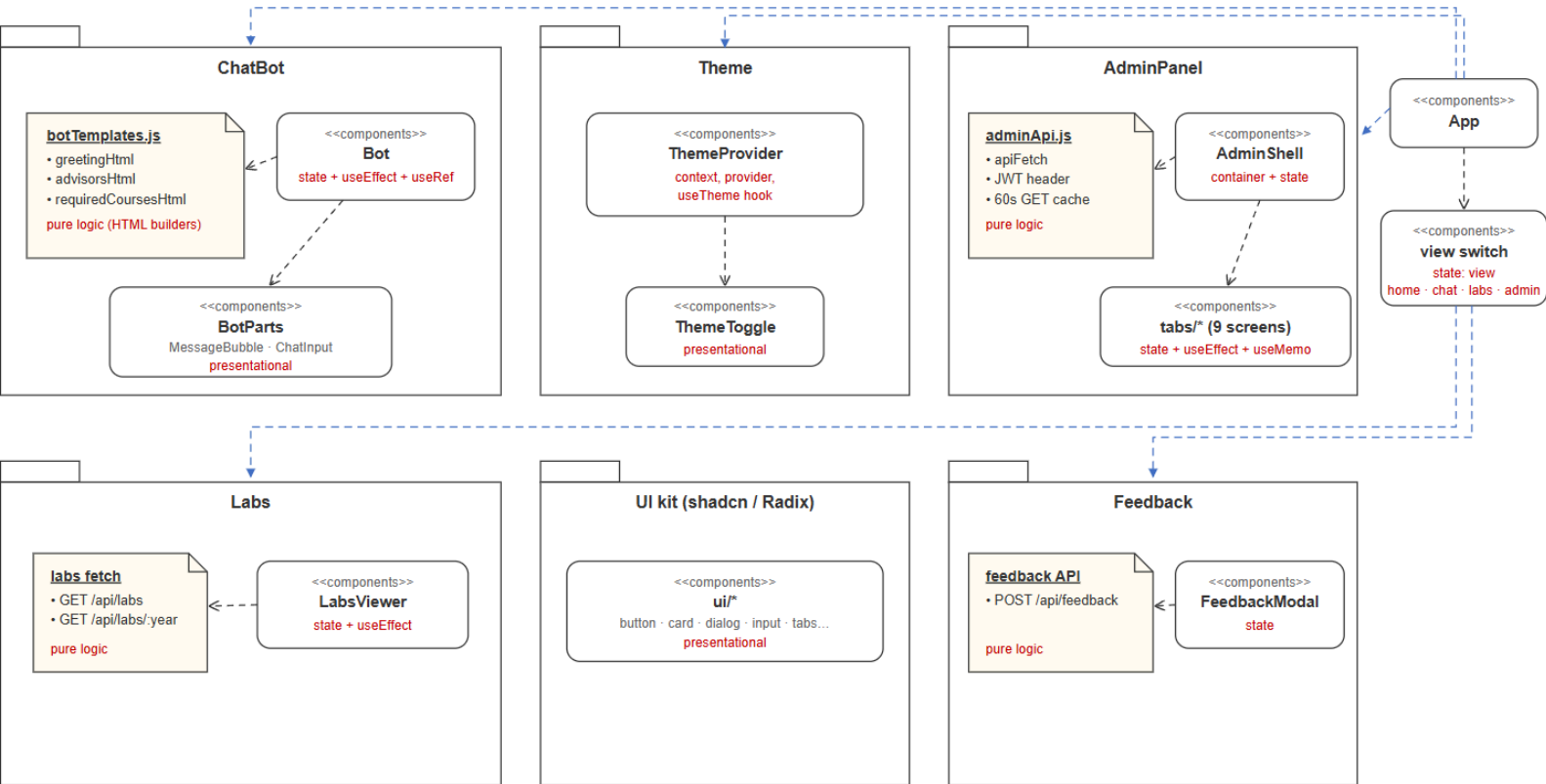
3. תהליך הפרויקט התחיל בראיון יחיד עם הלקוחה, שבו מיפינו את הצרכים האמיתיים של הסטודנטים ואת סוגי השאלות שחוזרות אליה שוב ושוב: שאלות על קורסים ותנאי קדם, על הרשמה, על מערכת המעבדות, על יועצים, וכן פניות של סטודנטים במילואים ופניות רגישות יותר. מתוך הראיון גם הבנו אילו מסמכים רשמיים הסטודנטים מחפשים ומה מקורות המידע שהלקוחה עצמה נשענת עליהם. בהמשך אספנו את המסמכים שמזינים את המערכת: שנתון הלימודים בפורמט וורד, מערכת המעבדות בפורמט אקסל, קובץ הנחיות ההרשמה, ומאגר הטפסים הרשמיים של המחלקה. המסמכים האלה הם הבסיס שממנו נבנה מאגר הנתונים, ולכן פיתחנו עבורם סקריפטים בפיתוח שמפרקים אותם אוטומטית לנתונים מובנים ומעלים אותם למסד. המערכת מורכבת מכמה מודולים מרכזיים: מנוע התשובות שמנתב כל שאלה ליכולת הנוכח, מודול הקורסים ותנאי הקדם, מודול המעבדות, מודול ההרשמה והיועצים, שכבת האחזור הסמנטי מעל התשובות המאושרות, ופאנל הניהול שדרכו מתחזקים את כל התוכן וצופים בנתוני השימוש. תוצרי הביניים של התהליך, ובהם תמלול הראיון עם הלקוחה והמסמכים שנאספו, מצורפים בנספח.

4. דיאגרמות:

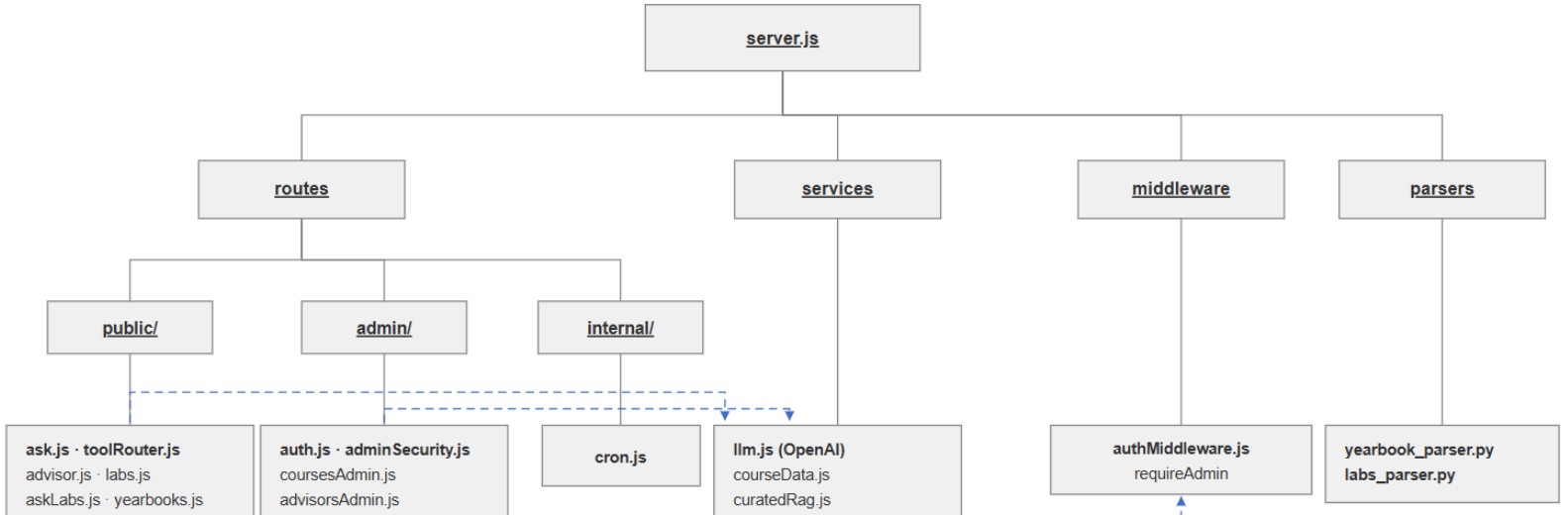
דיאגרמת מערכת:



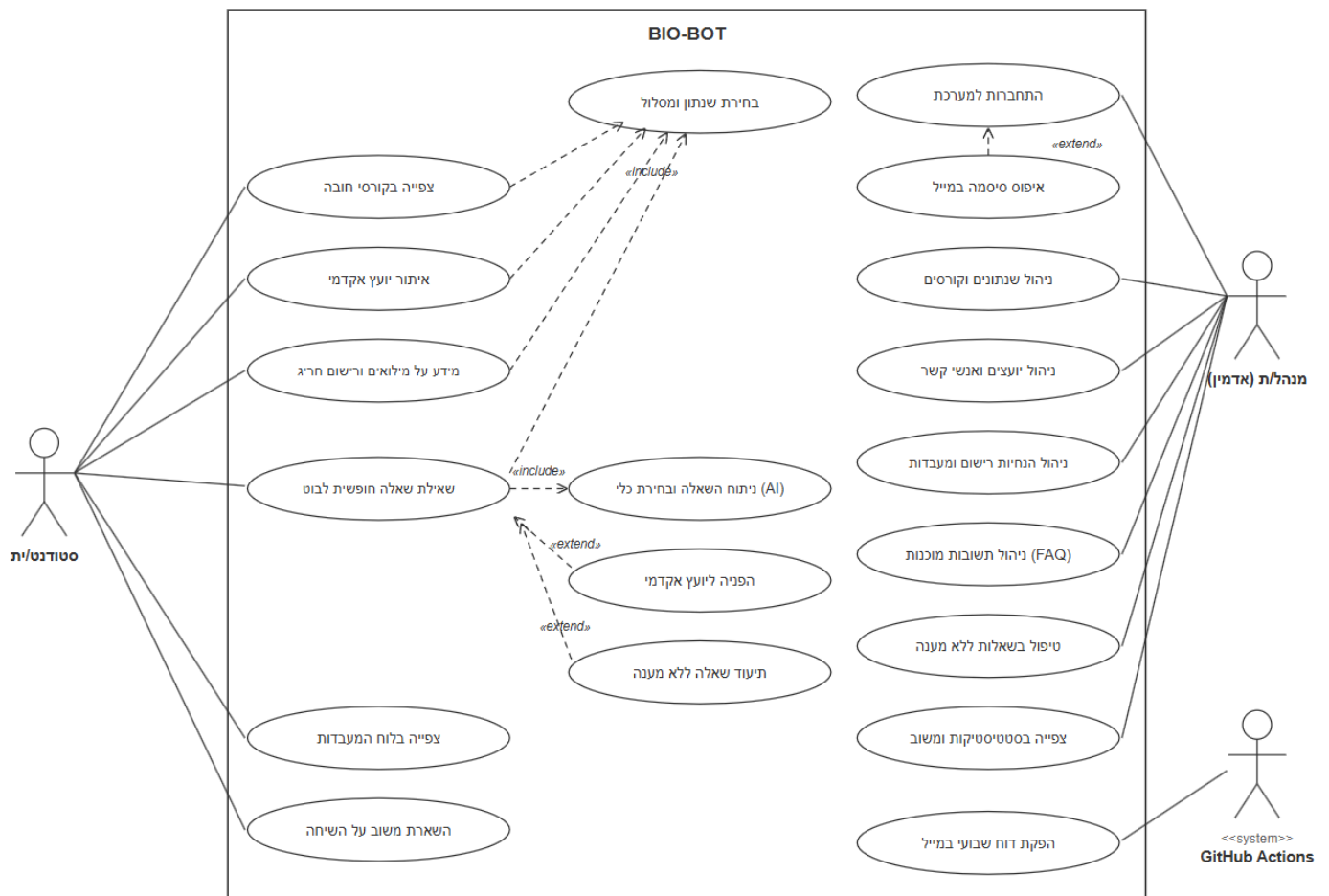
דיאגרמת לקוח:



דיאגרמת שרת:

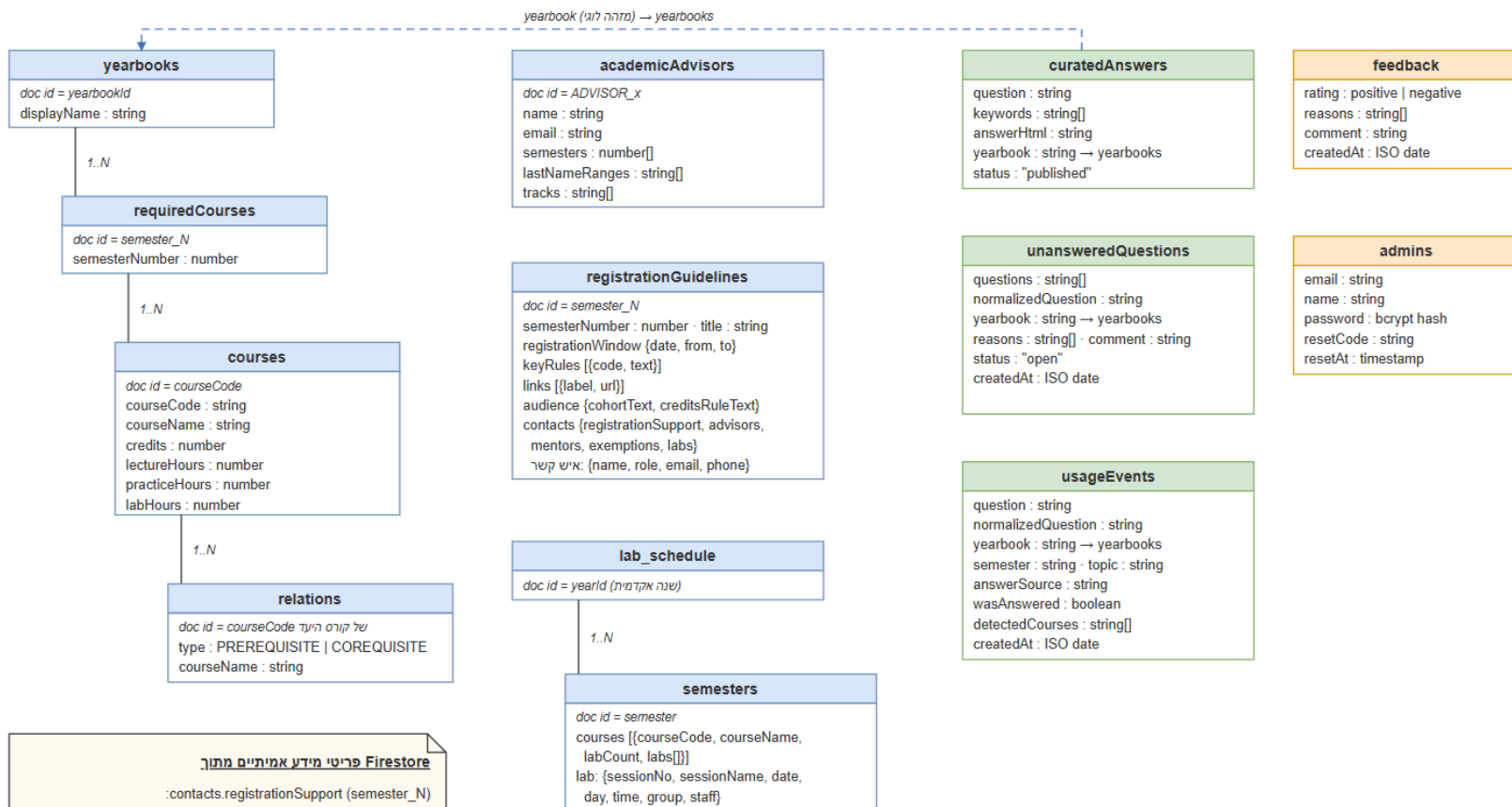


דיאגרמת Use-Case:



המחלקה להנדסת תוכנה ומערכות מידע

דיאגרמת DB עם פריטי מידע:



Firestore פריטי מידע אמיתיים מתוך

contacts.registrationSupport (semester_N)
 lasur@braude.ac.il · לילך יסעור קרוח · ראש המחלקה
 NataliaV@braude.ac.il · נטלי ופריק · מזכירת המחלקה
 mendaleyl@braude.ac.il · ליאורה מנדלי · עוזרת רמ"ח
 מולקולרית-תרופתית / מזון והסביבה
 lastNameRanges: "א-ת" : semesters: 1-8
 creditsRuleText: נ"ר 165
 relations.type: PREREQUISITE / COREQUISITE
 dean@braude.ac.il · 04-9901906 · דיקטט הסטודנטים

5. תוצרי מפגש הסטודיו:

א. חישוב ממוצע ציון SUS והתייחסות להערות :

לצורך הערכת השימושיות של מערכת **BIO-BOT 2.0**, נותחו תוצאות שאלון ה-SUS (System Usability Scale). השאלון כולל עשר שאלות סטנדרטיות, כאשר כל תשובה ניתנת בסולם של 1 עד 5.

ציון ה-SUS של כל משיב חושב לפי השיטה המקובלת:

בשאלות האי-זוגיות, שאלות 1, 3, 5, 7 ו-9, הופחת 1 מהדירוג שנתן המשתמש. בשאלות הזוגיות, שאלות 2, 4, 6, 8 ו-10, הופחת הדירוג שנתן המשתמש מ-5. לאחר מכן חוברו עשרת הערכים שהתקבלו, והסכום הוכפל ב-2.5, לקבלת ציון בטווח של 0 עד 100.

התקבלו 21 תשובות לשאלון. עבור משיב מספר 20 חסר נתון בשאלה מספר 6, ולכן תשובתו לא נכללה

בחישוב הממוצע. ציוני ה-SUS של 20 המשיבים שנכללו בחישוב היו:

55, 87.5, 77.5, 60, 100, 90, 72.5, 92.5, 80, 100, 90, 90, 90, 100, 95, 90, 97.5, 50, 100 ו-90.

סכום ציוני ה-SUS הוא **1,707.5**, ולכן הממוצע חושב כך:

$$\text{ממוצע SUS} = 1,707.5 \div 20 = 85.375$$

ממוצע ציון ה-SUS של המערכת הוא 85.375 מתוך 100. תוצאה זו מצביעה על רמת שימושיות גבוהה

מאוד ועל חוויית שימוש חיובית.

מהתגובות המילוליות עלו מספר נקודות חוזק. המשתמשים ציינו לחיוב את העיצוב המודרני והנקי של המערכת, את התמיכה במצב כהה ואת מהירות התגובה. בנוסף, המערכת נתפסה ככלי יעיל לאיתור מידע אקדמי, כגון סילבוסים ומידע על מעבדות, וככלי המסייע לסטודנטים בבניית מערכת שעות ובהתמצאות במהלך התואר.

לצד זאת, התקבלו גם מספר הצעות לשיפור. חלק מהמשתמשים ציינו כי יש לשפר את דיוק המענה של הבוט ואת מנגנון ה-RAG, כך שהמערכת תוכל לטפל במגוון רחב יותר של שאלות ולספק תשובות מדויקות יותר. כמו כן, הועלו הערות בנוגע לניגודיות של חלק מהצבעים בממשק, לעומס המידע בממשק הניהול, לגודלה של תיבת הזנת הטקסט ולתקלות נקודתיות ב-API של הצ'אט.

בעקבות המשוב, מתוכננים שדרוג של מודל השפה והרחבת בסיס הידע, שיפור הנגישות הוויזואלית באמצעות התאמת ערכת הצבעים, הגדלת תיבת הקלט והוספת הנחיות ברורות יותר למשתמשים חדשים.

6.א. תיעוד והסבר הפונקציות המרכזיות

עץ פונקציות

main.jsx

App.jsx — L

Bot.jsx — |

(startChat — |

(loadYearbooks — |

(sendMessage — L |

POST /api/ask — L |

(askHandler — L |

(detectGreeting — |

(classifyQuestion — |

(getAllPrerequisitesRecursive — |

(findContactsByQuery — |

(buildRagContext — L |

OpenAI API — L |

AdminShell.jsx — L

(apiFetch — L

Admin API — L

(requireAdmin — |

uploadAdmin.js — |

(execFile — L |

yearbook_parser.py — L |

reportService.js — L

— Brevo API / Nodemailer

startChat

קובץ: Bot.jsx

הפונקציה מאתחלת שיחה חדשה ומציגה את הודעת הפתיחה של הבוט. היא אינה מקבלת קלט ישיר ומעדכנת את מערך ההודעות של הצי'אט.

loadYearbooks

קובץ: Bot.jsx

הפונקציה טוענת את רשימת השנתונים הקיימים במערכת באמצעות בקשת GET לנתיב `api/yearbooks/`. התוצאה נשמרת במצב המקומי של הרכיב ומוצגת למשתמש. במקרה של כשל, המערכת מציגה הודעת שגיאה או משתמשת במערך ריק.

sendMessage

קובץ: Bot.jsx

הפונקציה שולחת את שאלת המשתמש לשרת ומנהלת את תצוגת ההודעות בצי'אט. הקלט כולל את שאלת המשתמש ואובייקט הקשר המכיל שנתון, סמסטר ומתווה מילואים. הפונקציה מוסיפה את השאלה למערך ההודעות, מציגה הודעת המתנה, שולחת בקשת POST ל-`api/ask/` ומציגה את התשובה שהתקבלה מהשרת. במקרה של כשל בתקשורת מוצגת הודעת שגיאה בצי'אט.

apiFetch

קובץ: adminApi.js

הפונקציה מבצעת קריאות HTTP מאומתות מלוח הבקרה של המנהל. היא מקבלת נתיב, סוג בקשה וגוף בקשה, שולפת את ה-JWT מתוך `sessionStorage`, מוסיפה אותו לכוותר `Authorization` ושולחת את הבקשה לשרת. כאשר השרת מחזיר קוד שאינו תקין, הפונקציה זורקת שגיאה.

askHandler

קובץ: `routes/public/ask.js`

זוהי פונקציית העיבוד המרכזית של שאלות המשתמש. היא מקבלת אובייקט JSON המכיל שאלה, שנתון, הקשר והיסטוריית שיחה, ומחזירה אובייקט בפורמט:

```
{ "...":html }
```

הפונקציה בודקת ברכות, שאלות מעבדה, כוונות רישום, תנאי קדם, אנשי קשר ומידע הקיים במנגנון ה-RAG. אם לא נמצאה תשובה מתאימה, השאלה נשמרת ב-`unansweredQuestions` והמשתמש מופנה ליועץ אקדמי.

detectGreeting

קובץ: `routes/public/ask.js`

הפונקציה מזהה אם השאלה היא ברכת פתיחה בלבד (למשל "שלום", "היי", "בוקר טוב") על ידי השוואת הטקסט המנומל לרשימת ברכות. היא מחזירה ערך בוליאני; כאשר מזהה ברכה, ה-`route` מחזיר ישירות הודעת פתיחה קבועה ללא קריאה למודל AI.

classifyQuestion

קובץ: `routes/public/ask.js`

הפונקציה מזהה את סוג השאלה, את הקורסים המוזכרים ואת כוונת המשתמש. היא בונה פרומפט סיווג (בהתחשב גם בהיסטוריית השיחה) וקוראת ל-`callLLMJson` מתוך `services/llm.js`, ומחזירה אובייקט JSON בפורמט `{ kind, courses, intent }`. פענוח ה-JSON מתבצע בתוך `services/llm.js` באמצעות `extractJson`, הכולל ניקוי גדרות ````json```` ונפילה חזרה לחילוץ הבלוק {...} הראשון.

getAllPrerequisitesRecursive

קובץ: [routes/public/ask.js](#)

הפונקציה שולפת באופן רקורסיבי את כל קורסי הקדם של קורס נתון. היא מקבלת קוד קורס ומזהה שנתון, בונה פעם אחת מפת קורסים, ופונה לתת-האוסף relations ב-Firestore עם סינון "type == PREREQUISITE". הפונקציה מזהה מעגלים באמצעות אוסף visited. עטיפת getAllPrerequisitesRecursiveCached מוסיפה מטמון של 5 דקות כדי לצמצם קריאות חוזרות למסד.

findContactsByQuery

קובץ: [routes/public/registration.service.js](#)

הפונקציה מבצעת איתור דטרמיניסטי של איש קשר לפי תפקיד (למשל "מי ראש המחלקה", "מזכירות") מתוך אנשי הקשר של הנחיות הרישום. היא מחזירה את האנשים התואמים, והתשובה מוצגת ישירות ללא קריאה למודל.

buildRagContext

קובץ: [routes/public/ask.js](#)

הפונקציה מרכיבה את הקשר הטקסטואלי שמועבר למודל השפה בקריאה הגנרטיבית (ב-answerOrRoute), כאשר האחזור הסמנטי לא מצא תשובה. היא מאגדת אנשי קשר, תקציר הנחיות רישום לסמסטר, רשימת הקורסים בשנתון לפי סמסטר, ומידע על מתווה המילואים של הסטודנט, ומקצצת את התוצאה ל-4000 תווים. הפונקציה אינה מבצעת embeddings.

ragCuratedAnswer

קובץ: [services/curatedRag.js](#)

הפונקציה מבצעת את האחזור הסמנטי מעל התשובות המאושרות. היא מחשבת embedding לשאלה באמצעות text-embedding-3-small, משווה אותו בדמיון קוסינוס מול ה-embeddings של curatedAnswers (מסוננות לשנתון), ומחזירה את התשובה בעלת הדמיון הגבוה ביותר אם היא עוברת את סף ה-0.78. אחרת מחזירה null והמערכת ממשיכה למסלול הגנרטיבי.

requireAdmin

קובץ: [authMiddleware.js](#)

זוהי פונקציית Middleware המגנה על נתיבי הניהול. היא בודקת את ה-JWT המופיע בכותר Authorization באמצעות jsonwebtoken ומשתנה הסביבה JWT_SECRET. אם חסר Token או שהוא אינו תקין, מוחזרת שגיאת 401. אם האימות הצליח, הבקשה ממשיכה לפונקציה הבאה. (הקוד אינו מחזיר 403).

Yearbook_parser.py

הסקריפט קורא קובצי שנתון בפורמט Word וממיר אותם לנתונים מובנים ב-Firebase. הוא מקבל נתיב לקובץ .docx, מזהה שנתון ותווית, ומחליץ קורסים וקשרי תנאי קדם ומקביל. הסקריפט משתמש בספריית python-docx ומופעל מהשרת באמצעות execFile, כדי לצמצם את הסיכון להזרקת פקודות.

ב. פירוט טכני :

1. ממשקי API שבהם נעשה שימוש

API פנימיים:

POST /api/ask

משמש לשליחת שאלת המשתמש לבוט. הקלט כולל שאלה, שנתון, הקשר והיסטוריית שיחה. הפלט הוא אובייקט המכיל תשובת HTML. הנתיב ציבורי ומוגן באמצעות הגבלת קצב בקשות.

GET /api/yearbooks

משמש לשליפת רשימת השנתונים הזמינים במערכת. הנתיב ציבורי ומחזיר מערך של שנתונים.

GET /api/advisor

משמש לאיתור יועץ אקדמי לפי מסלול, סמסטר ואות ראשונה של שם המשפחה. הנתיב ציבורי ומחזיר את פרטי היועץ המתאים.

POST /api/feedback

משמש לשמירת משוב אנונימי של סטודנט. הקלט כולל דירוג והערה מילולית, והפלט הוא סטטוס הצלחה.

POST /api/admin/auth/login

משמש לאימות מנהל. הקלט כולל כתובת מייל וסיסמה, והפלט כולל מזהה מנהל, שם ו-JWT. במקרה של פרטים שגויים מוחזרת שגיאת 401.

POST /api/admin/upload/yearbook

משמש להעלאת קובץ שנתון בפורמט .docx. הנתוב מוגן באמצעות JWT ומפעיל את סקריפט ה-Python לעיבוד הקובץ.

POST /api/internal/run-report

משמש להפעלת דוח שבועי אוטומטי. הנתוב מוגן באמצעות CRON_SECRET המועבר בכותרת .x-cron-secret

API חיצוניים :

OpenAI API

המערכת משתמשת במודל gpt-4o-mini לצורך סיווג כוונות, זיהוי רגש ויצירת תשובות, ובמודל

text-embedding-3-small לצורך אחזור סמנטי.

החיבור מתבצע באמצעות משתנה הסביבה OPENAI_API_KEY.

Brevo API

משמש לשליחת דוחות שבועיים והודעות דוא"ל. החיבור מתבצע באמצעות משתנה הסביבה

BREVO_API_KEY.

Firebase Admin SDK

משמש לביצוע פעולות קריאה וכתיבה במסד הנתונים Firestore מצד השרת בלבד.

2. קישור וחיבור ל-DB

מסד הנתונים של המערכת הוא Google Cloud Firestore. החיבור מתבצע בצד השרת באמצעות הספרייה `firebase-admin`. האתחול מתבצע ב-`server.js` בתצורת Singleton, כך שנוצר מופע אחד של מסד הנתונים המשמש את כל שירותי השרת. הגישה למסד הנתונים מתבצעת דרך Firebase Console ומוגבלת לחברי צוות מורשים ול-Service Account של השרת. צד הלקוח אינו ניגש ישירות למסד הנתונים. האוספים המרכזיים הם:

- `admins` - חשבונות מנהלים וסיסמאות מוצפנות.
- `yearbooks` - שנתונים, קורסים וקשרי תנאי קדם ומקביל.
- `academicAdvisors` - פרטי יועצים ומסלולים.
- `registrationGuidelines` - הנחיות רישום ואנשי קשר.
- `lab_schedule` - לוחות זמנים למעבדות.
- `curatedAnswers` - שאלות ותשובות מאושרות עבור RAG.
- `usageEvents` - נתוני שימוש אנונימיים.
- `feedback` - דירוגים והערות משתמשים.
- `unansweredQuestions` - שאלות שהמערכת לא הצליחה לענות עליהן.

קישור לדאטאבייס: <https://console.firebase.google.com/project/biobot2/firestore>

3. סביבות מיוחדות שהותקנו

צד הלקוח:

- React 19
- Vite 7
- Tailwind CSS 4
- Axios
- Lucide React
- MUI X Charts

צד השרת:

- Node.js
- Express 5
- Firebase Admin
- JSON Web Token
- bcrypt
- Multer
- Nodemailer

Python:

- `python-docx` לקריאת קובצי Word.
- `openpyxl` לקריאת קובצי Excel.

סביבות הרצה ושירותים נוספים:

- Vercel לאחסון צד הלקוח.
- Render לאחסון צד השרת.
- Google Cloud ו-Firebase לניהול מסד הנתונים.
- GitHub Actions להפעלת משימות מתוזמנות.
- קובצי `env` לשמירת מפתחות API ומשתני סביבה.

4. קטעי קוד ממקורות חיצוניים

רכיבי UI

רכיבי הממשק הבסיסיים הותאמו מתוך Radix UI ו-shadcn/ui.

רפרנסים:

<https://www.radix-ui.com/primitives>

<https://ui.shadcn.com>

פונקציית `cn()`

פונקציית עזר לשילוב מחלקות CSS, שנלקחה מתבנית ה-`utilities` של `shadcn/ui` והותאמה לפרויקט בקובץ:

`client/src/lib/utls.js`

מנגנון `Debounce`:

מנגנון ההשהיה בחיפוש הקורסים מומש באמצעות `setTimeout` ו-`useRef` בקובץ `BotParts.jsx`. המנגנון מבוסס על תבנית פיתוח מקובלת ב-`React` ולא הועתק ממקור מסוים.

5. פרומפטים שבהם נעשה שימוש בכלי AI

סיווג כוונות אקדמיות

קובץ: `services/llm.js`

"זהה את הקורסים המופיעים בשאלה ואת כוונת המשתמש: תנאי קדם, קורס מקביל או חיפוש. החזר JSON בפורמט `{ kind, courses, intent }`".

מענה RAG ו-Fallback

קובץ: `services/llm.js`

"ענה על השאלה הבאה בהתבסס רק על המידע המצורף. אם המידע אינו קיים, השב `NEED_ADVISOR`. אם השאלה אינה קשורה ללימודים, השב `OFF_TOPIC`".

זיהוי רגש

קובץ: `routes/public/ask.js`

"נתח את הטון הרגשי של ההודעה. אם המשתמש מביע מצוקה חמורה או עומס קיצוני, סווג כ-`DISTRESS`".

team_code	שם סוכן כדי שיוכל להשתתף ב- אורן SUS (נחשבת שהמערכת קלה לשימוש) ומאתרי SUS המעורבאים SUS (יחיד רצוח להשתמש במערכת זו לעיתים רחוקות לבצע את ההתקנות גם בזמן הפסקי ביעור את ההתקנות במאגרי רבה הנתון "סס" במחלק התקנות			
B20 Application with bot for bic6	5	3	4	2
B20 Application with bot for bic6	5	3	4	2
B20 Application with bot for bic6	6	7	למרות כל השימוש -	לחצונית - 5
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 3
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 4
B20 Application with bot for bic6	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 3
B20 Application with bot for bic5	5	5	למרות כל השימוש -	לחצונית - 5
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 2
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 2
B20 Application with bot for bic5	5	5	למרות כל השימוש -	לחצונית - 3
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 2
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 2
B20 Application with bot for bic6	6	6	למרות כל השימוש -	לחצונית - 1
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 5
B20 Application with bot for bic6	6	7	למרות כל השימוש -	לחצונית - 4
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 1
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 1
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 1
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 1
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 1
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 1
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 1
B20 Application with bot for bic7	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 1
B20 Application with bot for bic5	5	4	4	3
B20 Application with bot for bic6	7	למרות כל השימוש -	למרות כל השימוש -	לחצונית - 4

[illegible]

6. שמות משתמש וסיסמאות לכל סוג משתמש

מנהל מערכת

שם משתמש או מייל: braude.bio.bot@gmail.com

סיסמה: admin123

הרשאות: גישה מלאה ללוח הבקרה ולנתיבי `.api/admin/`

דרך התחברות: מסך ההתחברות בלוח הבקרה.

חשבון המנהל נשמר באוסף **admins**, והסיסמה נשמרת בצורה מוצפנת באמצעות **bcrypt**.

טוטוטו

שם משתמש: אין צורך בחשבון.

סיסמה: אין צורך בסיסמה.

הרשאות: שימוש בצ'אט, צפייה במידע אקדמי ושליחת משוב.

דרך התחברות: כניסה ישירה לממשק הראשי של המערכת.